

hospital-billing

Technical Design Document

Version 1.0 · Draft · 2026-09-24

CONFIDENTIAL

Document Control

Document Title	hospital-billing — Technical Design Document
Document Type	Technical Design Document
Version	1.0
Status	Draft
Classification	Confidential
Date	2026-09-24
Author(s)	Bitxn
Reviewer(s)	—
Approver(s)	—

Revision History

Version	Date	Author	Notes
1.0	2026-09-24	Bitxn	Initial version.

At a Glance

Project	hospital-billing
Files	7
Lines of code	459
Modules	2
Primary language	Python
Entry points	main.py
Analysis brain	oneport-managed (gemini)

Table of Contents

- 1. Executive Summary
- 2. System Overview
- 3. Architecture
- 4. Module Breakdown
- 5. Technology Stack
- 6. Data Flow
- 7. Setup & Deployment
- 8. Risks & Considerations
- 9. Appendix — File Inventory

1. Executive Summary

hospital-billing is a Python project comprising 7 files and roughly 459 lines of code, organised into 2 modules by responsibility. This document describes its structure, key components and how to run it, for engineers and reviewers onboarding to the codebase.

2. System Overview

A small, self-contained hospital billing engine. It turns a patient plus a list of rendered services into an itemized invoice — applying the service catalog, insurance coverage, discounts, and tax — and exposes it over a small REST API.

3. Architecture

The codebase is organised into 2 modules ((root), tests). Each module groups related files by responsibility; dependencies between modules define the flow of control across the system.

4. Module Breakdown

The system is organised into the following modules. Responsibilities are derived directly from the source tree, so this table doubles as a provenance map.

Module	Files	Responsibility
(root)	6	6 Python file(s): api.py, billing.py, database.py, main.py
tests	1	1 Python file(s): test_billing.py

5. Technology Stack

Languages detected across the codebase, by file count:

Language	Files
Python	7

6. Data Flow

Execution typically begins at main.py and flows inward through the module layers, with each module handling its own concern before returning results outward.

7. Setup & Deployment

Install the project's dependencies for its stack (Python), then run one of the entry points (main.py). See the repository README for exact commands.

8. Risks & Considerations

Areas warranting review include modules with many inter-dependencies (higher change cost) and any files lacking documentation. A follow-up compliance/security pass is recommended for regulated deployments.

9. Appendix — File Inventory

A condensed inventory of source files by module, for reference and traceability.

(root)

- `api.py` — `api.py` — a small Flask REST API over the billing engine.
- `billing.py` — `billing.py` — the billing engine (all the money math lives here).
- `database.py` — `database.py` — tiny JSON-file persistence for patients and invoices.
- `main.py` — `main.py` — CLI demo entryptpoint.
- `models.py` — `models.py` — the core data structures for the billing domain.
- `pricing.py` — `pricing.py` — the hospital service catalog.

tests

- `tests/test_billing.py` — `test_billing.py` — unit tests for the billing math.

Approval & Sign-off

This document requires the following approvals before it is considered final.

Role	Name	Signature	Date
Author	Bitxn		
Reviewer	—		
Approver	—		